全流程串联（一）：打通 Agent 主流程——chat / REST 的一次完整对话

Provider、ReAct、CLI、Notify、Tool、Memory、Sandbox、定时、Web Service——零件全做完了，每个模块的单元测试也都绿了。但"每个模块单测全绿"和"整台机器能转起来"是两码事。这一节和下一节不再开发新模块、不引入新概念，只做一件事：**把零件装成机器，让它真正跑起来。**

这是一节集成课。它的交付物不是某个新功能，而是"**主流程能从头到尾跑通**"这件事本身。集成课的价值不在多写几行代码，而在**它逼你暴露那些"我以为没问题"的假设**——那些藏在两个模块交接处、单测永远碰不到的坑。这一节先打通**人推链路**（有人发消息进来的一次完整对话，走 chat 或 REST），定时链路和 Demo 准备留到下一节。

一、本节目标：让 Agent 主流程"真能用"

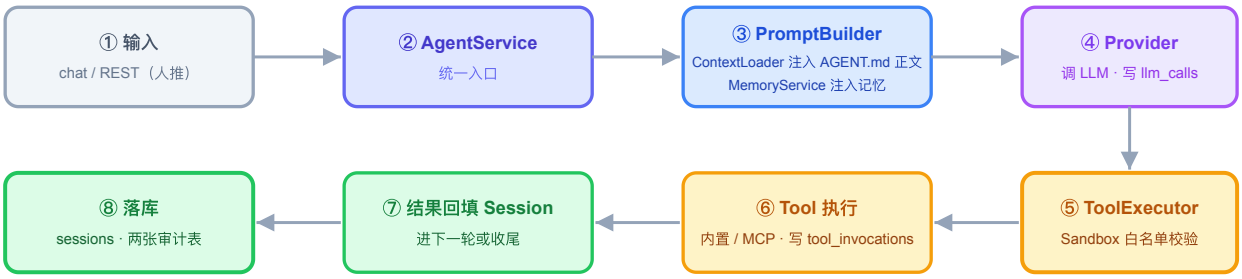
先把终点说清楚。这一节要达到的目的，一句话：

> **让 chat 或 REST 进来的一次完整对话，从头到尾走通"想 → 查 → 答 → 记账"这四步，把 Agent 的主流程从"单测绿"变成"真能用"。**

"真能用"不是凭感觉，是能验证的。给它一个具体标准——你随口问一句"今天北京天气怎么样，穿什么合适"，系统应当做到三件事：

1. **答得出来**：这句话会穿过八个环节，触发**两次大模型调用 + 一次工具调用**，最后回一段像样的穿衣建议；
2. **账记得对**：在 `sessions`（会话）、`llm_calls`（模型调用）、`tool_invocations`（工具调用）三张表里，留下**不多不少、刚好正确**的记录；
3. **三个入口看到的是同一份数据**：同一次对话，从命令行（CLI）、REST 接口、管理台网页去看，都是同一条会话——我们叫它"三面同源"。

这三件事同时成立，主流程才算打通；差一件，就是还没串好。



一次人推请求跑完，①~⑧ 每一站都要留下正确的痕迹

单个模块测试全绿 ≠ 链路能通——缝隙全都藏在两站之间的交接处

为什么现在做、用什么方法。 前面十一节，每个模块都是"单独看没问题"的：Provider 单独调得通、Memory 单独读写正常、Sandbox 单独拦得住。可一次真实对话要一口气穿过八个环节，只要有一处交接对不上——参数没传、格式不一致、假设不成立——整条链路就断在那儿。**坑几乎从不在模块内部，全在两个模块的接缝处。** 所以方法就一条，朴素但有效：**拿一次真实对话，从进到出走一遍；每经过一站，就核对它有没有在该留痕迹的地方，留下正确的痕迹。** 我们把这个动作叫"对账"。下面第二章，就是围绕这个目标要做的几件事。

二、围绕目标要做哪些

要让主流程从"单测绿"走到"真能用"，这一节做这么几件事：先确认零件都在位（2.1），再拉一次真实对话走一遍、逐张表对账（2.2），把走的过程中撞出来的坑一个个补上（2.3），把整台机器当着人的面跑起来（2.4），最后补一个对账要用、但上一节还没做的查询接口（2.5）。

2.1 先对表：确认每个模块都在位

动手之前，花十分钟核对一下：前面每个模块答应对外提供的能力，现在还在不在。这张表就是前面各节验收清单的浓缩：

模块（节）	就位标准
Provider（16）	显式映射路由正确；自动 tool 执行已关；`llm_calls` 成功失败都写
ReAct（17）	最大轮数兜底；每轮累积回 Session
CLI（18）	轻重命令分流；`chat` 能进能出
Notify（19）	`notify` 能推到 webhook；渠道未配置时明确报错
Tool（20）	九个内置 Tool 可调；三种来源统一成 `OryxTool`
Memory（21、22）	核心记忆始终在场；写入即读到（无缓存）
Sandbox（23、24）	三类白名单拦得住；违规写入 `tool_invocations`

模块（节）	就位标准
定时（25）	cron 到点触发；本地锁防重叠
Web Service（26）	10 端点全通；异常统一 JSON；管理台 Vue 构建产物在 `static/admin/` 、`/admin` 能打开

哪一行打不了勾，先回那一节修好再来。**带着一个坏零件装机器，最后查出来的全是假故障，白费劲。**

2.2 拉一次对话走一遍：怎么"对账"

挑一句最有代表性的话：在 `oryxos chat` 里问"今天北京天气怎么样，穿什么合适"。这一句会完整穿过八站（两次模型调用、一次查天气）。跑完之后，逐张表核对——这次对话应该留下、而且**只**留下这些痕迹：

- **`sessions`（会话表）**：1 条记录，里面的对话历史完整（用户问的、模型两次回复、一次天气结果）；
- **`llm_calls`（模型调用表）**：2 条（第一次让模型决定要不要查天气、第二次让它组织答复），两条的会话 id 一致、都成功、token 数不为零；
- **`tool_invocations`（工具调用表）**：1 条，就是那次 `http_get` 查天气，成功、耗时正常；
- **`MEMORY.md`（长期记忆）**：没动——这次对话不涉及记忆。如果它变了，说明有地方在乱写。

"该留的留下了"和"不该留的没多出来"一样重要。 工具被调了两次、`llm_calls` 冒出三条……这些"多出来的记录"，和"缺失的记录"一样都是 bug。

对完账，再从 Web 端用 `GET /api/v1/sessions/{id}` 查同一条会话，确认命令行和 REST 看到的是同一份数据。然后换 REST 入口把这句话重问一遍（`POST /sessions` 建会话、`POST /sessions/{id}/messages` 发消息），对账结果应当**完全一样**——"两个入口共用同一个引擎"不是口号，是能对出来的账。

2.3 会撞上的几个典型坑，怎么修

按上面的方法走一遍，大概率会撞上下面几类问题。它们各自在前面某一节埋下了引线，到串联时集中引爆——这正是集成课要抓的东西：



① 启动日志里 "Found 0 JPA repositories"。 18 节讲过：主类上的 `scanBasePackages` 带不动 `@EnableJpaRepositories` / `@EntityScan`。单跑 storage 模块的测试没事，一从 CLI 主类启动就炸——典型的"接缝处"问题。修法：在启动类上显式写清这两个注解的 `basePackages`；验收标准也写死——启动日志里 "Found N JPA repository interfaces" 的 N 必须大于 0。

② 同一个工具被调了两次。 这是 16 节"坑二"没关干净的全局表现：`tool_invocations` 里同一个工具同一时刻两条记录，或者天气被查了两遍。修法回 16 节——确认 Spring AI 的自动执行是关的，工具的执行权只在 `ToolExecutor` 一处。

③ 会话 id 两个入口对不上。 会话 id 是"渠道 + 用户 + Profile"三者拼出来的（技术方案 9.2）。如果 CLI 和 Web 各自拼了一遍、格式差一个分隔符，同一个用户就会分裂成两条互不相认的历史。修法：拼 id 的逻辑只留在 `SessionManager` 一处，所有入口只把三样原料传进去、绝不自己拼。

④ 少了个 Bootstrap 文件，却被悄悄跳过。 `.oryxos/` 里少了 `SOUL.md`，如果 `ContextLoader` 选择"跳过不吭声"，Agent 的人格设定就悄悄丢了——症状是"回答风格不对"，这种软故障最难查。修法：缺文件至少打 WARN 日志；Profile 里明确引用了的文件如果缺失，直接报错。

⑤ 审计漏了失败的那一半。 成功路径的审计一般没问题，**最容易漏的是失败路径**：Provider 超时那次、Sandbox 拦下那次、工具执行抛异常那次，是不是每次都落了一条 `success=false` 的记录？把这三种失败各造一次，逐条对账。

2.4 跑起来给人看：启动 Web Service 与管理台

前面都在对"痕迹"，这一步第一次要把整台机器**当着人的面跑起来**。这里要先说清一件事：Web Service 和管理台（web manager）是**同一个进程的两张脸**—— `oryxos serve` 起的那个 Spring，同时对外提供 `/api/v1/**` 的 REST 接口和 `/admin` 的管理台页面，不是两个服务、不用起两个进程。

这是一个开发任务，不是"打开个网页"。 管理台是 Vue 写的（26 节交付，跟官网首页同一套 Vue 3 + Vite），不是随手写的静态 html，而是要**构建**出来、由 Spring 托管：前端 `npm run build` 的产物落在 `oryxos-web/src/main/resources/static/admin/`，Spring 把 `/admin` 指过去，并且对 `/admin/**` 没命中的路径回落到 `index.html`（这样前端路由刷新子页面不会 404，`/api/v1/**` 不受影响）。所以"启动管理台" = 先把前端构建进 jar，再 `serve`——这条构建链就是本节要串通、并写进 README 的东西。

两个前置（不满足就是空转）： ① `export DEEPSEEK_API_KEY=...`——因为 26 节已经排除了 `OpenAiAutoConfiguration`，只要这一个 key 就能起（如果它还嚷着要 `spring.ai.openai.api-key`，回 26 节补排除）；② 前端已经构建进 `static/admin/`。

启动步骤（一条命令同时给出 REST 和管理台）：

```
# 1. 构建管理台前端（Vue+Vite）——若已用 frontend-maven-plugin 绑进 mvn，这步并进第 2 步
cd oryxos-web/src/main/frontend && npm ci && npm run build && cd -
#    产物落在 oryxos-web/src/main/resources/static/admin/

# 2. 打整包（fat JAR 里已含管理台静态资源）
mvn clean package -DskipTests

# 3. 启动 Web Service（同一进程同时托管管理台）
export DEEPSEEK_API_KEY=your-key
java -jar oryxos-boot/target/oryxos-boot-*.jar serve --port 8080

# 4. 两张脸各验一下
curl localhost:8080/api/v1/health        # REST 通
open  http://localhost:8080/admin         # 管理台（Vue）能打开、各页渲染真实数据
open  http://localhost:8080/swagger-ui    # 接口文档
```

这一步就是"三面同源"的收口：在管理台里点开"会话"，看到的应当**正是**刚才在命令行 / REST 里那次"天气穿衣"对话——同一份数据、同一个引擎的第三个视图。开发调试时也可以 `npm run dev` 起前端热更、把接口代理到 8080，但**发布形态是"一个 jar 托管一切"**，README 要按发布形态写。

2.5 补一个对账要用的查询接口：列出会话

对账和"三面同源"要想**自动验证**，前提是每一样痕迹都得有一个 HTTP 接口查得到。拿本节要断言的东西，逐条对着 26 节交付的 10 个端点查缺：

要断言的	靠哪个接口	现状
对话产生了会话、能在列表里看到	`GET /api/v1/sessions`（列出会话）	**缺——本节补**

要断言的	靠哪个接口	现状
某次会话的完整往来	`GET /api/v1/sessions/{id}`	26 节已有
记忆写进去、查得到	`GET /api/v1/memory`	26 节已有
工具清单查得到	`GET /api/v1/tools`	26 节已有
Profile / 运行状态	`GET /api/v1/profiles` 、 `/info`	26 节已有

记忆和工具的查询接口 26 节都做了，直接用；**唯一缺的是"列出会话"**——26 节只做了按 id 查一条，没有列表接口，管理台"会话"页也因此只能空着。本节把它补上：

- **`GET /api/v1/sessions`** —— 列出会话，按最后活跃时间倒序，默认返回最近 100 条的**摘要**（会话 id、Profile 名、渠道、用户、状态、创建时间、最后活跃时间、消息条数），**不带**完整对话正文以免列表过大；可选参数 `?status=active` 。
  - **要改的地方**： `SessionManager` 加一个 `List<Session> listRecent(int limit)` （实现走 `SessionRepository` ， `JpaRepository` 自带的 `findAll` 已够用，按活跃时间排序取前 N）； `SessionApiController` 加一个不带 `{id}` 的 `@GetMapping` ，再加一个会话摘要 DTO。做法沿用 26 节扩展 `archive()` / `readAll()` 的老路子。
  - 这个接口顺带把 26 节管理台"会话"页的空缺补上了——之前只能按 id 查、列不出来，正是这里补全后接上真实列表。

> 记忆的关键词检索 `MemoryService.recall(keyword)` 目前只在 ReAct 内部用。本节 harness 用 `GET /memory` 拿全文断言就够，**不强制**给它单开接口；以后若要让运营侧按关键词查记忆，再加 `GET /api/v1/memory/recall?keyword=` 也不迟。

三、怎么验收：把对账固化成一个测试

手工对账查完就忘；这一节要把它**固化成一个能一键重演的集成测试** `HumanTriggerFlowIT` （打 `@Tag("integration")` ，需要真 key、CI 默认跳过、手动或按需跑）。harness 设计得好不好，标准只有一条：**它替你把"对话答得出 + 记忆查得到 + 工具查得到 + 三面同源"这几件事自动断言一遍，而不是每次靠人肉再查。** 目标的三个标准，正好对应三根支柱：

支柱一 · 对话答得出（主流程跑通）

1. `POST /api/v1/sessions` 建会话 → 断言拿到会话 id；
2. `POST /api/v1/sessions/{id}/messages` 发"今天北京天气怎么样，穿什么合适" → 断言**答复非空**、不是异常信封；
3. `GET /api/v1/sessions/{id}` → 断言历史里有：用户消息、两次模型回复、一次工具结果；
4. `GET /api/v1/sessions` （本节新增）→ 断言这条会话**出现在列表里**、摘要字段对得上；
5. 查库对账： `llm_calls` 恰 2 条、 `tool_invocations` 恰 1 条（ `http_get` ）、全部成功——不多不少。

支柱二 · 记忆查得到（写得进、读得出）

1. 发一条会触发 `save_memory` 的消息（如"记住：我在北京，怕冷"）→ 跑完；
2. `GET /api/v1/memory` → 断言返回内容里**查得到刚写入的事实**（含"北京"）；
3. 再开一轮新会话问"我在哪个城市" → 断言答复用上了这条记忆（跨对话记得住，这就是 Demo 二的雏形）。

支柱三 · 工具查得到（工具清单对外可见）

1. `GET /api/v1/tools` → 断言返回内置工具清单、**数量大于 0**、包含 `http_get` / `save_memory` 等关键工具；
2. 和 Profile 里声明的工具集对照，确认工具注册表和对外看到的一致。

兜底 · 失败也要记账 + 三面同源

- **失败路径**（接 2.3 的第 ⑤ 条）：把 Provider 挂掉、Sandbox 拦截、工具抛异常各造一次，断言审计表里都有 `success=false` 的记录、系统本身不崩；
- **三面同源**：命令行和 REST 各把"天气穿衣"走一遍， `GET /sessions/{id}` 两次数据一致；管理台"会话"页（靠新增的 `GET /sessions`）能列出、点开就是这次对话。

无 key 也能跑：用 mock provider 把全链路搬进 gate

`HumanTriggerFlowIT` 要真 key + 网络，只能手动跑。为了让"全链路能不能跑通"这件事**在 gate 里也能自动判**，加一个**独立的 mock provider**——挂在显式映射表的 `mock` 名下（宪法 III），不连任何真实模型、不需要 key。它按脚本驱动一次确定性的 ReAct：第一轮请求一次 `save_memory` 工具调用（**真写 `MEMORY.md`**，给全链路一个可观测的"文件写入"行为），第二轮（PromptBuilder 已把工具结果渲染成 `tool: …` 行）直接返回最终答复。**只有"模型"是假的**，ReActLoop / ToolExecutor / Memory / Session / 审计全部走真实路径。

于是有了两层无 key 自测，都进 gate、随 `mvn verify` 自动跑：

- **`MockProviderFlowTest`（手工装配、快）**：直接拼出 ReActLoop + mock provider + 真实工具/记忆/会话，一条"记住…"消息 → 断言 **ReAct 两轮**、`save_memory` **恰调一次**、`MEMORY.md` **记下事实**、**会话历史完整**、审计 `llm_calls` ×2 / `tool_invocations` ×1；再用同一批服务起 `/sessions` 、 `/memory` 、 `/tools` 端点，确认 **console 查得到这条链路留下的数据**。
- **`MockAgentE2ETest`（真启动整台服务）**：`@SpringBootTest` 起**真实 HTTP 端口 + SQLite**，从测试里 `POST /sessions` 建会话、`POST …/messages` 发对话，再把会话 / 记忆 / 工具 / 审计全查回来——跟人手动点一遍完全一样，只是自动化了。为让它 hermetic，`OryxOsRuntime` 的工作区根目录支持用系统属性 `oryxos.root` 覆盖（默认仍 `.oryxos`）：测试把根指向临时工作区、SQLite 指向临时库、provider 覆盖成 `mock`。

手动怎么测：无 key 用 mock 走一遍

自动化之外想亲手验一遍，用 `mock` provider（不烧 key、不联网）。

前置：`bin/start.sh 8080` 起服务；`config/application.yml` 里有 `- name: mock`；`.oryxos/profiles/` 下有个 `provider: mock` 的 Agent（如 `mock-agent`，`tools: [save_memory]`）。

命令行——看全链路：

```
B=http://localhost:8080/api/v1
# 1) 建会话（用 mock-agent，不烧 key）
curl -s -X POST $B/sessions -H 'Content-Type: application/json' -d
'{"profile":"mock-agent"}'
#    → data.sessionId = web:default:mock-agent
# 2) 发一条"记住…"（mock 会驱动一次 save_memory + 两轮 ReAct）
curl -s -X POST $B/sessions/web:default:mock-agent/messages \
  -H 'Content-Type: application/json' -d '{"content":"记住：我喜欢喝美式咖啡"}'
# 3) 查会话历史（user / assistant / tool / assistant 四条）
curl -s $B/sessions/web:default:mock-agent | python3 -m json.tool
# 4) 会话列表   5) 长期记忆（含"美式咖啡"）   6) 工具清单（含 save_memory）
curl -s $B/sessions | python3 -m json.tool
curl -s $B/memory
curl -s $B/tools | python3 -m json.tool
```

浏览器——管理台（先发一两条对话再开）：`http://localhost:8080/admin/`（首次 `Cmd+Shift+R` 硬刷新）。侧边栏「会话」看到刚才那条、「长期记忆」看到"美式咖啡"、「Tool」含 `save_memory`、「Provider」有 `mock`——CLI/REST/管理台三面同源。

换真模型：把 `profile` 换成 `default`（走 deepseek，需 `config/application.yml` 已填 key），消息随便问，如"今天北京天气怎么样，穿什么合适"。

验审计（可选）：`sqlite3 oryxos.db "select tool_name,success from tool_invocations where session_id='web:default:mock-agent';"`（`llm_calls` 同理，每轮对话 2 条）。

一句话：**`mock-agent` 无 key 验全链路，`default` 验真模型；两者数据都进同一个管理台。**

验收清单（每条都成立才算达标）

- **对话**：命令行、REST 两个入口各走一遍，答复非空，逐表对账一致、不多不少；
- **会话列表**：`GET /api/v1/sessions` 能列出刚才的会话，管理台"会话"页不再是空占位；
- **记忆**：触发 `save_memory` 后，`GET /api/v1/memory` 查得到写入的事实，新会话能用上；
- **工具**：`GET /api/v1/tools` 返回工具清单、数量大于 0、含关键工具；
- **审计**：三种失败路径各造一次，审计表里都有 `success=false`，系统不崩；
- **装配**：启动日志里 JPA repositories 数量大于 0、所有 Profile 加载成功、引用的 Bootstrap / Skill 文件都在；

- **不回退**：`mvn clean verify` 全绿（含 `MockProviderFlowTest` / `MockAgentE2ETest` 两个无 key 全链路测试）；`HumanTriggerFlowIT` 打 `@Tag("integration")`、需真 key 手动跑。

本节交付物

- **代码**：`SessionManager.listRecent(int)`（oryxos-core 接口扩展）+ `JpaSessionManager` 实现；`SessionApiController` 新增 `GET /api/v1/sessions`（会话摘要列表）+ 摘要 DTO；**`MockChatModel`**（独立 mock provider，挂 `mock` 名下、无 key 驱动全链路）+ 工厂接线与配置校验放行；`OryxOsRuntime` 工作区根支持 `oryxos.root` 系统属性覆盖（为整机测试指向临时工作区，默认 `.oryxos` 不变）。
- **测试（harness）**：`HumanTriggerFlowIT`（`@Tag("integration")`，真 key）覆盖三根支柱 + 失败路径 + 三面同源；**`MockProviderFlowTest`**（gate 内、无 key，手工装配全链路 + console 数据）；**`MockAgentE2ETest`**（gate 内、无 key，`@SpringBootTest` 真启动整台服务，HTTP 建会话/发对话/查回会话·记忆·工具·审计）；`MockChatModelTest` 钉 mock 脚本行为。
- **前端**：管理台"会话"页接上 `GET /api/v1/sessions`，从占位改成真实列表。
- **配置**：`config/application.yml.example` 增内置 `mock` provider 示例（无需 key）。
- **文档**：README 端点表补上 `GET /api/v1/sessions`；本节"手动怎么测"给出无 key 走 mock 的命令行 + 管理台流程。

到这里，人推主干就通了：一句话进来，走完想、查、答、记账，每一站的痕迹都对得上，会话、记忆、工具都查得到，三个入口看到的是同一份数据。下一节把另外半边接上：**定时链路**（到点自动触发 → ReAct → notify 推送）、重启不失忆、多个 Agent 并存——那是两个 Demo 上场前的最后一轮查缺补漏。